This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

ProScale: Proactive Autoscaling for Microservice with Time-varying Workload at the Edge

Ke Cheng, Sheng Zhang, *Senior Member, IEEE*, Chenghong Tu, Xiaohang Shi, Zhaoheng Yin, *Student Member, IEEE*, Sanglu Lu, *Member, IEEE*, Yu Liang, Qing Gu

Abstract—Deploying microservice instances on the edge device close to end users can provide on-site processing thus reducing request response time. Each microservice has multiple instances that can process requests in parallel. To achieve high processing efficiency, the number of these instances is scaled according to the workload, which is also known as autoscaling. Previous studies of microservice autoscaling in the edge computing environment lack in-depth consideration of time-varying workload, they assume that the workload of each microservice always depends on that of its upstream. However, through an analysis of Alibaba's microservice trace with hundreds of millions of records, we find that the assumption is impractical thus hurting autoscaling effectiveness. To solve this problem, we propose ProScale, a prediction-driven proactive autoscaling framework for microservices at the edge. ProScale proactively forecasts the workload for each individual microservice per timeslot. Then it utilizes an efficient online algorithm to leverage the predicting results to determine the instance number for each microservice jointly with making placement decisions. For each microservice instance deployed on the edge device, ProScale handles burst requests using a designed offloading strategy. In addition, ProScale can also balance the load for multiple instances of each microservice. Extensive trace-driven experiments show that ProScale has great scalability. It can reduce average response time by 96.7% and resource usage by 96.5% compared with existing strategies and designed baselines.

Index Terms—Microservice, workload prediction, proactive autoscaling, edge computing

1 Introduction

Microservice is an architectural style that breaks down an application into multiple smaller microservices which collaborate to accomplish user requests [1]. It has many advantages compared with the conventional monolithic architecture. For example, each microservice has multiple containerized instances processing calls in parallel. These instances are lightweight and can be quickly started and terminated, greatly improving the ease of deployment [2]. Besides, each microservice can be independently developed, tested, updated, and maintained, which enhances the software maintainability and speeds up the application development. Therefore, microservice has become the mainstream application architecture in recent years [3]. It has been widely employed by Internet giants such as Google, Amazon, Netflix, and Alibaba [4], [5], [6], [7].

Inspired by the success of microservice, nowadays people also deploy microservice instances in the edge computing environment to provide on-site processing to reduce request response time [8], [9]. The edge computing environment consists of the cloud data center and multiple edge devices which are typically servers that are deployed in proximity to base stations and can host microservice instances. Previous works [10], [11] have studied the placement of microservice instances in the edge computing environment and achieved excellent performance, but there still remain challenges. The major challenge is to ensure efficient on-site processing for each microservice in face of the time-varying workload. In this paper, we define the workload of a microservice as the number of Calls received by all the instances of the microservice Per Second (CPS).

To solve this challenge, the autoscaling mechanism is proposed to adjust the number of instances for each microservice according to the dynamically varying workload. Fig. 1 describes the microservice autoscaling in the edge computing environment. As shown in Fig. 1, an application composed of four microservices is orchestrated at the edge. Multiple instances of each microservice are deployed not only on the cloud but also on edge devices. Since the workload of microservices 2 and 3 increases, new instances of them are started to provide sufficient parallel processing capability. As the workload of microservice 4 decreases, one of its instances is terminated to save the resources of the edge device.

Several prior works [12], [13] have studied microservice autoscaling at the edge, but they generally lack in-depth consideration of the time-varying workload. They always make the assumption that the workload of each microservice is always proportional to that of its upstream and accordingly make autoscaling decisions. However, through an analysis of the Alibaba microservice trace [14], we find that the assumption made by these works does not hold in practice. Moreover, we also identify that the misapplication of such an assumption causes ineffective resource allocation

- *Ke Cheng, Sheng Zhang, Chenghong Tu, Xiaohang Shi, Sanglu Lu, and Qing Gu are with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China. E-mail: {ketonmi, xiaohangnju}@outlook.com, {sheng, sanglu, guq}@nju.edu.cn, chenghongtu@smail.nju.edu.cn*
- *Zhaoheng Yin is with the department of electronic and computer engineering of the Hong Kong University of Science and Technology, Hong Kong 999077, China. E-mail: zhaoheng.yin@connect.ust.hk*
- *Yu Liang is with the School of Computer and Electronic Information/School of Artificial Intelligence, Nanjing Normal University, China. Email: liangyu@njnu.edu.cn*

Manuscript received August 2, 2022; revised January 13, 2023.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

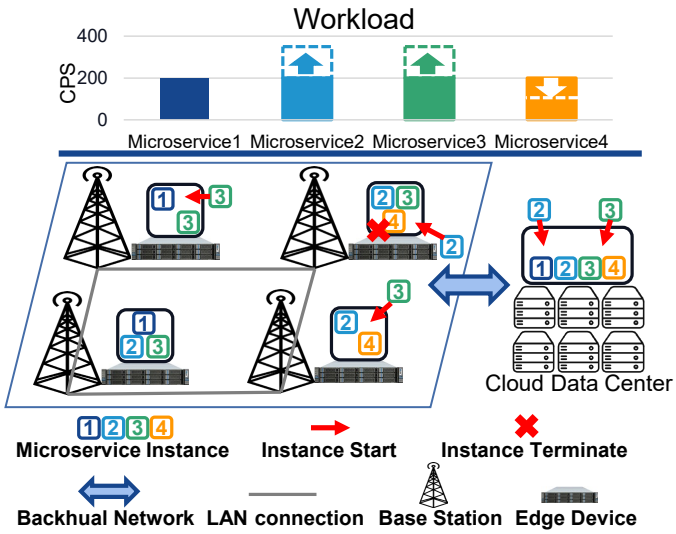


Fig. 1: Microservice autoscaling at the edge. As the workload of microservices 2 and 3 increases, their new instances are started. As the workload of microservice 4 decreases, one of its instances is terminated.

for each microservice, which hurts the performance of autoscaling and hence affects the request response time. At the same time, we find that the workload of most microservices is predictable while burst requests are difficult to be precisely predicted through the analysis.

Considering these findings and the limitations of previous works, we propose to proactively predict the workload for each microservice and leverage the predicting results to determine the instance number for each microservice jointly with making placement decisions for the instances at the edge. Since the prediction is more accurate than the unrealistic assumption made by previous works, autoscaling can adapt to the time-varying workload and make better placement decisions to improve performance.

However, there are three main challenges that have to be addressed. **Challenge for Fast Predicting**: A long predicting time hurts the timeliness of predicting results, causing delayed autoscaling, thus leading to performance issues. Therefore, a fast predicting method must be proposed. **Challenge for Effective Scaling**: The heterogeneous edge computing environment and complex microservice architecture pose challenges in determining the number of instances for each microservice and making placement decisions for the instances. **Challenge for Burst Handling**: Burst requests are hard to be predicted, and therefore specifically designed mechanism is required to handle burst requests.

To resolve the above challenges, we proposed ProScale, a prediction-driven proactive autoscaling framework for microservices in the edge computing environment. It utilizes a predictor to periodically forecast the workload for all the microservices with an accurate and fast predicting method. Its microservice scaler then leverages a heuristic online algorithm supported by the workload predicting results to determine the number of instances for each microservice jointly with making placement decisions in the edge computing environment. Moreover, It also has a burst handler and a load balancer for each instance deployed on edge devices, which can further boost the processing performance. Our main contributions are as follows.

- We analyze the Alibaba microservices dataset and find that the workload of a microservice does not depend on that of its upstream, and the workload of most microservices is predictable while burst requests are unpredictable. Based on these discoveries, we design ProScale composed of a predictor, a microservice scaler, a burst handler, and a load balancer to proactively scale microservices at the edge.
- The predictor of ProScale leverages the accurate and fast Simple Moving Average (SMA) method to periodically forecast the workload for each microservice and the effectiveness of SMA is validated by comparing it with four state-of-the-art workload predicting methods and two industrial forecasting tools.
- A problem formulation of microservice scaling at the edge is presented to minimize the request response time considering both multidimensional resource constraints and the workload demand. Its NP-hardness is proved and an efficient online algorithm is utilized by the scaler of ProScale to resolve it.
- The burst handler of ProScale is designed to avoid a large number of sudden requests piling up on edge devices by the edge-cloud collaboration, and the load balancer of ProScale is proposed to make full use of the computing capability of powerful edge devices while avoiding overloading weak devices.
- Extensive trace-driven experiments are conducted to validate the superiority of ProScale. The experimental results show that ProScale outperforms the existing strategies and baselines in request response time and resource usage, and all of the four components contribute to performance improvement.

The rest of this paper is organized as follows. Section 2 introduces representative related works. In section 3, we demonstrate the research background and our motivation which comes from an analysis of the Alibaba microservice trace. The system model and the problem of microservice scaling at the edge are formulated in Section 4, where the problem is proved to be NP-hard. We describe the design of ProScale in Section 5 where the predictor, microservice scaler, burst handler, and load balancer are depicted in detail. The superiority of ProScale is validated in Section 6. Finally, we conclude this paper in Section 7.

2 Related Works

We summarize some prior studies on related topics by the following categories and highlight their drawbacks compared with our work.

2.1 Microservice Autoscaling on the Cloud

Ding *et al*. [15] and Hossen *et al*. [16] make autoscaling decisions only based on the utilization of CPU while Zhang *et al*. [17] and Kwan *et al*. [18] adjust the instance number of each microservice not only according to the utilization of CPU but also memory. In addition to considering the utilization of resources, Qiu *et al*. [19] Yu *et al*. [20], Gias *et al*. [21] and Choi *et al*. [22] also focus on the request response time. Besides, Yang *et al*. [23] and Yang *et al*. [24] pay attention not

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

only to request response time but also to request queuing time and processing time of each microservice instance. Furthermore, Gotin *et al*. [25] investigate which of the four performance metrics, CPU utilization, queue delay, queue growth, and queue length, is more suitable for threshold-based microservice autoscaling. Apart from performance metrics, there are also other studies that scale microservices directly according to the workload. Abdullah *et al*. [26], [27], Iqbal *et al*. [28] and Prachitmutita *et al*. [29] respectively leverage the elastic net regression, linear regression, and long short-term memory to forecast the workload for each microservice and proactively make autoscaling decisions. Moreover, Iqbal *et al*. [30] jointly consider workload and request response time and dynamically allocate resources for each microservice with an unsupervised learning method.

However, these works only focus on microservice autoscaling on the cloud data center without considering the heterogeneous edge computing environment.

2.2 Microservice Placement at the Edge

Liu *et al*. [31], Wang *et al*. [32] and Jang *et al*. [33] design algorithms to optimize the response time of microservice-based applications at the edge. Besides, the user mobility in the edge computing environment is also considered in existing works [34], [35], [36] where the customized genetic algorithm and reinforcement learning based methods are respectively proposed to reduce the request response time. Moreover, Gu *et al*. [37], [38] consider containerized microservice and make placement decisions for each container layer. Furthermore, Zhao *et al*. [10], Pallewatta *et al*.[11] and Deng *et al*. [39] deploy multiple instances at the edge for each microservice with meta-heuristic algorithms and the branch-and-bound method while Lv *et al*. [40] and Fu *et al*. [41] implement systems to reduce the tail latency of the microservice-based applications.

However, the number of microservice instances and the placement schema remains fixed over a long time in these studies, which makes them incapable to accommodate the time-varying workload.

2.3 Microservice Autoscaling at the Edge

Li *et al*. [12]. exploit an adaptive discrete binary particle swarm optimization algorithm to solve the problem of microservice autoscaling at the edge with request response time as the optimization target. They proactively estimate the workload for each microservice and then accordingly make autoscaling decisions. He *et al*. [13] propose a greedy-based algorithm to utilize the estimated workload to proactively determine the number of instances for each microservice jointly with making placement decisions.

However, both of them estimate the workload for microservices based on the impractical assumption that the workload of a microservice always depends on that of its upstream.

3 Background and Motivations

In this section, we first explain the basic concepts of the microservice architecture and the time-varying workload with an example of a microservice-based video analysis application. Then, we demonstrate the limitation of the performance metrics adopted by previous studies of microservice autoscaling and highlight the advantages of making autoscaling decisions based on the workload. Moreover, through an analysis of the Alibaba microservice trace, we confirm that the α-assumption made in previous works of microservice autoscaling at the edge is unrealistic. Furthermore, we point out the performance issue caused by adopting such an impractical assumption, which motivates us to forecast the workload for each microservice.

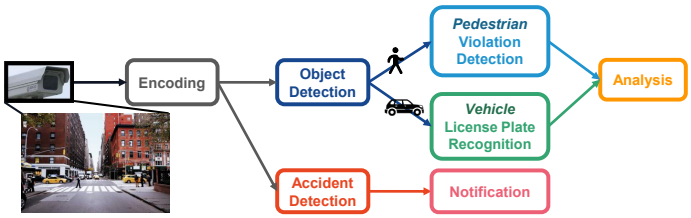


Fig. 2: Microservice-based video analysis application.

3.1 Microservice and Time-varying Workload

A microservice-based application consists of multiple microservices. As cyclic dependency is considered a bad practice in the microservice architecture [42], the microservice-based application is always depicted as a Directed Acyclic Graph (DAG) where vertexes denote microservices and edges represent call relationships between microservices.

Fig. 2 shows a microservice-based video analysis application composed of seven microservices where *Encoding (ECD)* encodes the crossroad video uploaded by the camera and calls *Object Detection (OD)* to detect objects in each frame, then the detected pedestrians and vehicles are respectively transmitted to *Pedestrian Violation Detection (PVD)* and *Vehicle License Plate Recognition (VLPR)* for processing, and the processed results are sent to the *Analysis (ALY)* for generating statistics. In addition, the encoded video is also sent to the *Accident Detection (AD)*. If a traffic accident is detected, the *Notification (NTF)* will be called.

All the microservices which call microservice m are named m's Upstream Microservice (UM), and m is named their Downstream Microservice (DM). For example, *ECD* is the UM of *OD* and *AD*, and *ALY* is the DM of *PVD* and *VLPR*. Besides, we name the microservice that is directly called by end users the entry microservice, and its workload is named User Request Rate (URR). *ECD* is the entry microservice of the video analysis application shown in Fig. 2. Furthermore, an instance of microservice m is a container with programs of m running inside.

The workload of a microservice is defined as the number of calls received by all the instances of the microservice per second. A microservice always has multiple instances running simultaneously for processing requests in parallel and the autoscaling mechanism adjusts the number of instances for each microservice to accommodate the time-varying workload. In the above example, the workload of *PVD* and *VLPR* is higher during the day than at night as the number of detected pedestrians and vehicles increases, so new instances of these two microservices will be started to handle the raised workload.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

3.2 Limitation of Previous Performance Metrics

Previous proactive autoscaling methods always forecast performance metrics like CPU utilization, average request processing time, and average request queuing time for each microservice to detect the microservices to be overloaded/underloaded, and start/terminate instances for them. However, these performance metrics indirectly reflect the workload through statistics generated during the application execution, which leads to delayed reflection of the workload variation and causes performance issues.

Performance metrics are produced according to the statistical information generated during the application execution. However, when the workload changes, the relevant statistics do not change immediately, but after the microservice instances have already been executing for some time, which makes performance metrics reflect the workload with a delay. Previous proactive autoscaling methods leverage historical data of the performance metric to predict the incoming situation and hence make autoscaling decisions. When the predicting result indicates that a microservice is about to be overloaded, it is often because, in the historical data, the performance metric of the microservice shows an upward trend and is close to the threshold of being judged as overloaded. However, because of the delayed reflection of performance metrics, when the historical data of the metric is close to the overload threshold, the microservice is possibly already overloaded, thus causing a large number of queued requests and affecting the request response time.

To solve this problem, previous proactive autoscaling methods set the overload threshold at a low value to further advance autoscaling. However, the solution is not suitable for the edge computing environment, because edge devices are rare. If the overload threshold of the performance metric is set low, overprovision will occur and hence result in wasting precious resources of edge devices. If the overload threshold is not set to low, delayed autoscaling will occur due to the delayed workload reflection. Since edge devices do not have as much computing capability as the cloud, the delayed autoscaling may result in overload and a large number of requests will pile up in edge devices, thus inducing a long request queuing time, and the request response time will be greatly affected.

Directly predicting workload can completely avoid the delayed workload reflection caused by the above-mentioned performance metrics. Proactive autoscaling strategies can predict workload for all the microservices, and compare the predicting result with the processing capacity for each microservice, then start/terminate instances for the microservices whose processing capacity is about to be underprovisioned/overprovisioned.

3.3 Impractical α-assumption

Previous works [12], [13] of microservice autoscaling at the edge make autoscaling decisions according to the workload. However, they made the unrealistic α-assumption and calculate the workload of each microservice based on the workload of its UM, which hurts the effectiveness of autoscaling.

The α-assumption supposes that for each call relationship, there exists a stable call ratio α ($\alpha > 0$) making the workload of the callee microservice always α times of the UM's workload. That is, given an α and a call relationship composed of a microservice m and the corresponding UM, if $\alpha > 1$, the α-assumption indicates that when the UM is called, it will call m α times, and $\alpha \leq 1$ represents that m will be called once after the UM has been called $\frac{1}{\alpha}$ times. In [12], the call ratio α of each call relationship is a fixed value and does not change over time. In [13], the call ratio α is calculated by the Average Call Frequency Coefficient (ACFC). Although ACFC is not a constant value, it tends to be stable over time. Apart from the above two studies, there are also works [22], [30], [43] of microservice resource management on the cloud that implicitly make the α-assumption.

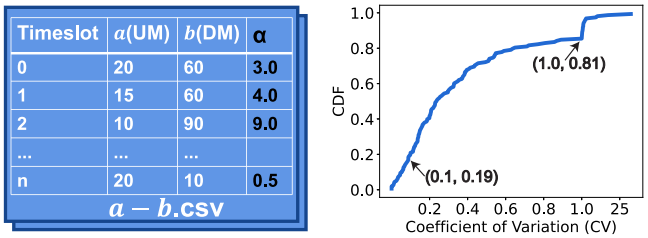


Fig. 3: UM-DM file content and CDF of CV_α. (a) Content of the UM-DM file. (b) Cumulative distribution of CV_α of each UM-DM file.

However, we confirm that the α-assumption is impractical through an analysis of Alibaba microservice trace with hundreds of millions of records. At first, we extracted the workload sequence of each microservice with only one UM and the workload sequence of its UM into the same file. Next, for each UM-DM file, we calculate the α in each timeslot by dividing UM's workload at the timeslot by DM's workload at the timeslot. The timeslot length of the trace data is one minute, and the workload at a timeslot is the average of the workload in that timeslot.

Fig. 3a. shows an example of the content of a UM-DM file. For each UM-DM file, we calculate the coefficient of variation (CV) of α. It is calculated by

$$CV_\alpha = \frac{\sigma_\alpha}{\mu_\alpha}, \tag{1}$$

where σ_α and μ_α respectively denote the standard deviation and the mean of α in all timeslots. A small CV_α indicates that the α is stable while a big CV_α represents that the α is dramatically varied. If the α-assumption in [12] is true, the α in every timeslot will be constant and the σ_α will be 0, thereby the CV_α will be 0 as well. However, there is no case where the CV_α is 0. Besides, if the CV_α is less than 0.1, we can almost consider that the α remains stable and the α-assumption in [13] holds. Even so, as shown in Fig. 3b, only less than 20% of the UM-DM files are consistent with the assumption, which confirms that the α-assumption is unrealistic. Moreover, we can also find the CV_α of almost 20% UM-DM files is bigger than 1.0, which represents that the α is dramatically varied. Furthermore, there are even cases where the CV_α exceeds 25, which further presents that α-assumption is inconsistent with the actual situation.

The untruth of α-assumption can be intuitively explained by the example of the video analysis application shown in Fig. 2. The workload of *OD* is equal to the frame

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

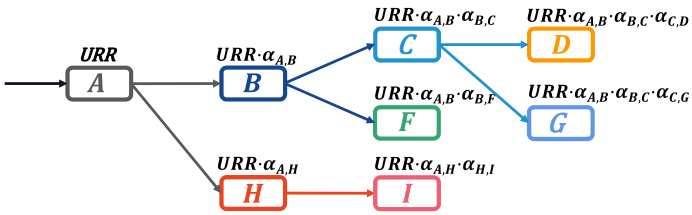


Fig. 4: Based on the impractical α-assumption, the workload for each microservice can be calculated by that of its UM. $\alpha_{U,M}$ represents the assumptive stable call ratio α between the microservice M and its UM U.

rate of the uploaded video while the workload of *PVD* and *VLPR* is varied according to the number of detected pedestrians and vehicles respectively. If there is no vehicle in the video, the workload of *VLPR* will be 0 no matter how high is the frame rate. Consequently, the workload of each microservice is jointly determined by the user request rate, request content, and program logic. We can not simply assume that the workload of a microservice always depends on that of its UM.

As shown in Fig. 4, prior works that make α-assumption can calculate the workload of each microservice by that of its UM. In this way, the workload of all the microservices can be calculated through the URR. When the URR varied, they accordingly adjust the number of instances for each microservice according to the calculated workload. However, since the α-assumption is not true, there will be a non-negligible gap between the calculated workload and the actual workload, which further causes the number of deployed instances for each microservice to be inconsistent with the actual required instance number. If the deployed instance number is less than the required number, a great number of requests will pile up in the queue thus increasing the request response time. If the deployed number is greater than the actual demand, the resource utilization of instances will be low thus wasting resources of edge devices.

To solve this problem, we propose ProScale which forecasts the workload for each microservice and uses the predicting results to scale microservices at the edge.

4 System Model and Problem Formulation

In this section, we consider the microservice scaling at the edge in the tth timeslot. We first describe the system model and formulate the problem of microservice scaling at the edge. Then, we prove that the problem is NP-hard. Relevant notations used in the system model and problem formulation are presented in Table 1 where the index t of the notations is omitted for the sake of simplicity.

4.1 System Model

We detail the models of the edge computing environment and the microservice-based application. In addition, the instance placement schema and the instance makespan are modeled as well.

4.1.1 Edge Computing Environment

The edge computing environment is heterogeneous. The heterogeneity comes from network and edge devices [44].

TABLE 1: Notations

Notation	Description
$\mathcal{E}$	Set of edge devices
C	Cloud data center
$\mathcal{D}$	Set of devices in the edge computing environment
Γ_d	CPU core number of $d \in \mathcal{D}$
Θ_d	Memory size of $d \in \mathcal{D}$
Φ_d	Storage volume of $d \in \mathcal{D}$
$b_{d,d'}$	Bandwidth between $d, d' \in \mathcal{D}$
$l_{d,d'}$	Propagation latency between $d, d' \in \mathcal{D}$
$T^{tran}_{d,d',w}$	Time for transferring w units data from d to $d' \in \mathcal{D}$
$\mathcal{M}$	Set of microservices
$\mathcal{U}_m$	UM set of $m \in \mathcal{M}$
m_{start}	Virtual microservice that receives user request
γ_m	Required CPU core number of $m \in \mathcal{M}$
θ_m	Required memory size of $m \in \mathcal{M}$
ϕ_m	Required storage volume of $m \in \mathcal{M}$
$\Delta_{m,m'}$	Data volume transmitted between $m, m' \in \mathcal{M}$
$T^{proc}_{m,d}$	Processing time of the instance of $m \in \mathcal{M}$ deployed on $d \in \mathcal{D}$
$r_{m,d}$	Processing rate of the instance of $m \in \mathcal{M}$ deployed on $d \in \mathcal{D}$
ψ_m	Workload of $m \in \mathcal{M}$
$n_{m,d}$	Instance number of $m \in \mathcal{M}$ deployed on $d \in \mathcal{D}$
$\mathcal{N}$	Overall placement schema
$\mathcal{I}_m$	Instance set of $m \in \mathcal{M}$
$\mathcal{I}$	Instance set of the microservice-based application
x_i	Microservice of instance $i \in \mathcal{I}$
y_i	Device where instance $i \in \mathcal{I}$ is deployed
L_i	Makespan of instance $i \in \mathcal{I}$

The network heterogeneity includes edge-to-edge network heterogeneity and edge-to-cloud network heterogeneity [45]. The former represents that the communication bandwidth and propagation latency between different edge devices are different, and the latter implies that the edge devices are connected via the high-speed Local Area Network (LAN) while communicating with the cloud through the backhaul network.

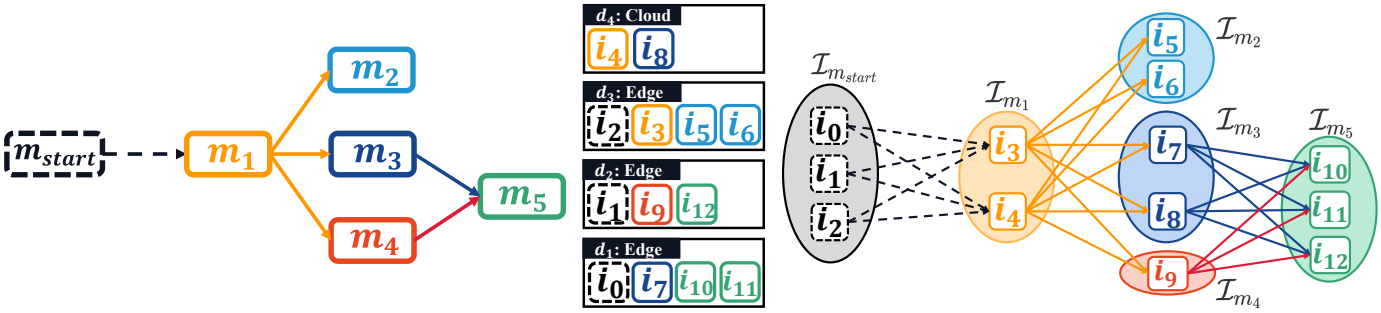
The device heterogeneity includes resource capacity heterogeneity and computing capability heterogeneity [39]. The former represents that different edge devices have different CPU core numbers, memory sizes, and disk volume, and the latter implies that different edge devices process the same task at different speeds due to their different operating systems, hardware, and their special optimization technologies.

We model the edge computing environment as a device set $\mathcal{D}$ which consists of a set of heterogeneous edge devices $\mathcal{E}$ and a cloud data center C, that is, $\mathcal{D} = \mathcal{E} \cup \{C\}$. Given a device $d \in \mathcal{D}$, it is characterized by its multi-dimensional resources in terms of CPU, memory, and storage, which are respectively symbolized as Γ_d, Θ_d, and Φ_d.

The bandwidth and propagation latency between device d and d' are separately denoted as $b_{d,d'}$ and $l_{d,d'}$. In this paper, we assume that $b_{d,d'} = b_{d',d}$, $l_{d,d'} = l_{d',d}$. The transmission time required for transferring w units of data from d to d' is calculated by

$$T^{tran}_{d,d',w} = \begin{cases} l_{d,d'} + \frac{w}{b_{d,d'}} & \text{if } d \neq d' \\ 0 & \text{if } d = d'. \end{cases} \quad (2)$$

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429



(a) (b) (c)

Fig. 5: Illustration of the placement schema. (a) The application to be scaled. (b) Instances of the microservices and their deployed devices which are determined by the placement schema. (c) Instance set of each microservice and the call relationships between these instances.

4.1.2 Microservice-based Application

A microservice-based application is depicted as a DAG where the vertexes represent microservices and edges denote call relationships. Its microservice set is symbolized as $\mathcal{M}$ and the data volume transmitted between $m, m' \in \mathcal{M}$ is denoted as $\Delta_{m,m'}$. Given a microservice m, $\mathcal{U}_m$ represents its UM set. Each microservice m is described by its required resources in terms of CPU, memory, and storage, which are respectively symbolized as γ_m, θ_m, and ϕ_m.

Due to the computing capability heterogeneity of the edge devices, the processing rate of each microservice instance deployed on different devices is diverse. $T^{proc}_{m,d}$ is used to describe the time required for processing one call for an instance of microservice m deployed on device d. In this paper, we suppose that each call is handled by one thread and multiple CPU cores can process calls parallelly. Therefore, the processing rate $r_{m,d}$ for the instance of microservice m deployed on the device d can be calculated by

$$r_{m,d} = \frac{1}{T^{proc}_{m,d}} \cdot \gamma_m. \tag{3}$$

Moreover, we add one virtual microservices m_{start} to the microservice set $\mathcal{M}$. The m_{start} is responsible for forwarding user requests to instances of the entry microservice. The m_{start} is deployed on every edge device. Furthermore, we add an edge from m_{start} to the entry microservice and set the data volume transferred between them to the size of the user request.

4.1.3 Placement Schema

The placement schema is expressed by $n_{m,d} \in \mathbb{N}$. $n_{m,d} = k$ indicates that there are k instances of microservice $m \in \mathcal{M}$ that are deployed on device $d \in \mathcal{D}$. Moreover, $\mathcal{N}$ represents the overall placement schema which contains the $n_{m,d}$ for every microservice m and device d. As the m_{start} is deployed on every edge device, for $\forall d \in \mathcal{E}$, $n_{m_{start},d} = 1$.

Given a placement schema of the application depicted in Fig. 5a where $n_{m_{start},d_1} = 1$, $n_{m_{start},d_2} = 1$, $n_{m_{start},d_3} = 1$, $n_{m_1,d_3} = 1$, $n_{m_1,d_4} = 1$, $n_{m_2,d_3} = 2$, $n_{m_3,d_1} = 1$, $n_{m_3,d_4} = 1$, $n_{m_4,d_2} = 1$, $n_{m_5,d_1} = 2$, and $n_{m_5,d_2} = 1$. Fig. 5b shows the instances and their deployed devices determined by the placement schema, and Fig. 5c groups these instances into the instance set of microservices and plots the call relationships between instances. Each instance in Fig. 5b and Fig. 5c has the same color as its corresponding microservice in Fig. 5a.

Given a placement schema, the instance set of microservice $m \in \mathcal{M}$ is defined as $\mathcal{I}_m$, and the instance set of the microservice-based application is denoted as $\mathcal{I} = \bigcup_{m \in \mathcal{M}} \mathcal{I}_m$. For each instance $i \in \mathcal{I}$, $x_i \in \mathcal{M}$ denotes its microservice, and $y_i \in \mathcal{D}$ represents the device where it is deployed. As shown in the example in Fig. 5, $\mathcal{I}_{m_1} = \{i_3, i_4\}$, $\mathcal{I}_{m_3} = \{i_7, i_8\}$, $x_{i_6} = m_2$, and $y_{i_6} = d_3$.

4.1.4 Instance Makespan

The makespan of an instance $i \in \mathcal{I}$ is recursively defined as

$$L_i = \max_{i' \in \mathcal{I} \wedge x_{i'} \in \mathcal{U}_{x_i}} \{L_{i'} + T^{Tran}_{y_{i'}, y_i, \Delta_{x_{i'}, x_i}}\} + T^{Proc}_{x_i, y_i}, \forall i \notin \mathcal{I}_{m_{start}}. \tag{4}$$

Specifically, $L_i = 0, \forall i \in \mathcal{I}_{m_{start}}$.

4.2 Problem Formulation and Analysis

The problem of microservice scaling at the edge is formulated for minimizing the request response time while adhering to the resource constraints of each device and the workload demands of each microservice. Moreover, we prove that the problem is NP-hard and underline its difference from other problems of microservice scaling, microservice placement, task offloading, and network function placement.

4.2.1 Problem Formulation

Our objective is to find an optimal placement schema to minimize the maximal request response time which is defined as the maximal instance makespan while adhering to the resource constraint for each device and satisfying the workload demand for each microservice. We omit the time required for transferring the response message to the end user because the data volume of the response message is always very small and the transferring time is short and can be ignored. Given the workload of each microservice $m \in \mathcal{M}$ which is symbolized as ψ_m, the problem of microservice scaling at the edge can be formulated as

$$\mathbb{P}: \min \max_{i \in \mathcal{I}} L_i \tag{5}$$

subject to:

$$\sum_{d \in \mathcal{D}} n_{m,d} \geq 1, \forall m \in \mathcal{M}. \tag{6}$$

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

$$\sum_{m\in\mathcal{M}} n_{m,d}\cdot\gamma_m \le \Gamma_d, \forall d\in\mathcal{D}, \tag{7}$$

$$\sum_{m\in\mathcal{M}} n_{m,d}\cdot\theta_m \le \Theta_d, \forall d\in\mathcal{D}, \tag{8}$$

$$\sum_{m\in\mathcal{M}} n_{m,d}\cdot\phi_m \le \Phi_d, \forall d\in\mathcal{D}, \tag{9}$$

$$\sum_{d\in\mathcal{D}} n_{m,d}\cdot r_{m,d} \ge \psi_m, \forall m\in\mathcal{M}. \tag{10}$$

In the above formulation of $\mathbb{P}$, Eq. (6) denotes that each microservice must have at least one instance deployed. Eq. (7), (8), and (9) respectively represent the resource constraints of each device in terms of CPU, memory, and storage. Moreover, Eq. (10) denotes the workload demand.

4.2.2 Problem Complexity Analysis

Proposition. *$\mathbb{P}$, the problem of microservice scaling at the edge which aims at minimizing the maximal request response time is NP-hard.*

Proof. We first describe the well-known NP-hard problem, 0-1 knapsack problem [46]. Then we reduce the 0-1 knapsack problem to a special case of $\mathbb{P}$ and show $\mathbb{P}$ is also NP-hard.

Given the item set $\mathcal{M}$ with the value v_m and weight w_m of each item $m\in\mathcal{M}$. The 0-1 knapsack problem wants to determine which items to be packed in order to maximize the overall value while meeting the total weight constraint B, that is,

$$\mathbb{Q}: \max \sum_{m\in\mathcal{M}} x_m\cdot v_m \tag{11}$$

subject to:

$$\sum_{m\in\mathcal{M}} x_m\cdot w_m \le B, \tag{12}$$

where $x_m\in\{0,1\}$ represents whether the item m is packed.

We prove that $\mathbb{P}$ is NP-hard by reducing the 0-1 knapsack problem to a special case of $\mathbb{P}$. The special case doesn't consider data transmission and microservices of the application to be scaled are serially connected into a chain. Besides, there are only two devices at the edge, an edge device d_{edge} with limited resources and the cloud C with unlimited resources. For each microservice m, $r_{m,C}\ge 1$ and $r_{m,d_{edge}}\ge 1$, and the workload of each microservice is 1, which means that only one instance of each microservice needs to be deployed to meet the workload demand. We introduce $x_m\in\{0,1\}$ to denote that the instance of microservice m is deployed on the edge device ($x_m=1$) or the cloud ($x_m=0$). Therefore, the maximal request response time is the makespan of the instance of the tail microservice in the chain, which can be calculated by summing up the processing time of all the microservice instances. Moreover, we remove the resource constraints of the edge device in terms of CPU and memory. The special case of $\mathbb{P}$ can be written as follows.

$$\mathbb{P}': \min \sum_{m\in\mathcal{M}} ((1-x_m)\cdot T^{proc}_{m,C} + x_m\cdot T^{proc}_{m,d_{edge}}) \tag{13}$$

subject to:

$$\sum_{m\in\mathcal{M}} x_m\cdot\phi_m \le \Phi_{d_{edge}}. \tag{14}$$

We can reduce $\mathbb{Q}$ to $\mathbb{P}'$ by the following encoding. $\forall m\in\mathcal{M}$, $T^{proc}_{m,C}=0$, $T^{proc}_{m,d_{edge}}=-v_m$, $\phi_m=w_m$ and $\Phi_{d_{edge}}=B$. We can solve $\mathbb{Q}$ exactly by giving these inputs to $\mathbb{P}'$. Therefore, $\mathbb{P}$ is NP-hard.

4.2.3 Difference from Other Problems

In addition to the related works of microservice scaling [12], [13] and placement [31], [33], [34], [35], [36], [10], [11], [39], [40], [41] at the edge which model the microservice-based application as a DAG, we also collect several representative studies of task offloading [47], [48], [49] and network function placement [50], [51], [52], all of which aim to place tasks and network functions of DAG structure on multiple devices. We analyze the difference between their problems and $\mathbb{P}$ from the following three aspects.

Difference in the optimization objective. The optimization objective of our problem is in the form of recursion. However, only the problems in [31], [48], [50] are in the form of recursion, while the problems in other studies are in the form of summation.

Difference in the constraints. Our problem has multi-dimensional resource constraints for each device and the workload demand constraints for each microservice. However, several studies of microservice placement [41], network function placement [50], [52], and all the studies of task offloading [47], [48], [49] do not have resource constraints while some studies of microservice placement [31], [35], [10], [40] only have single-dimensional resource constraints for each device.

Difference in the decision variable. The decision variables in our problem are non-negative integers. Moreover, the decision variables in our problem can not be converted to 0-1 variables which represent the placement schema of each instance because the instance number for each microservice can not be determined in advance due to the computing capability heterogeneity of edge devices. Given a microservice m and its predicted workload, if all its instances are deployed on a powerful device with a high processing speed, the number of its instances will be small, if all its instances are deployed on a weak device with a low processing speed, the number of its instances will be large. The instance number of each microservice is tightly coupled to the placement schema and can not be determined beforehand, and therefore the decision variables in our problem can not be converted to 0-1 variables. However, only the decision variables in [13], [10], [39], [40] are also non-negative integers while others are 0-1 variables.

5 ProScale: Solution Description

To achieve proactive autoscaling at the edge, we should finish microservice scaling at the beginning of each timeslot by solving the problem $\mathbb{P}$. However, the workload of each microservice, ψ_m in the timeslot can not be obtained until the end of the timeslot. Therefore, we propose ProScale which leverages an accurate predictor to predict the workload in the timeslot for each microservice, and then we can solve $\mathbb{P}$ by replacing ψ_m with the predicted workload.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

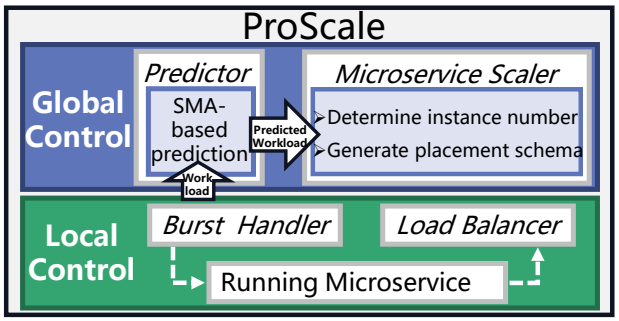


Fig. 6: Design overview of ProScale

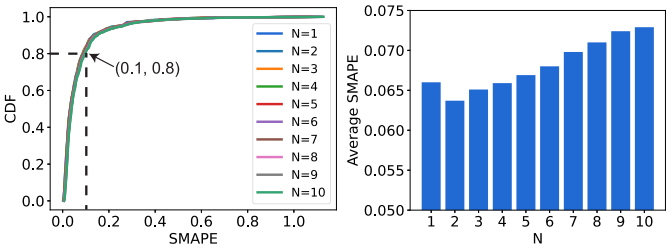


Fig. 7: Predicting results of SMA with different N. (a) Cumulative distribution of SMAPE. (b) Average SMAPE.

As shown in Fig. 6, ProScale has two sub-modules, a global control module and a local control module. The global control module is composed of a *Predictor* and a *Microservice Scaler*. At the beginning of each timeslot, it collects the workload in the last timeslot from all the deployed instances, and *Predictor* uses the collected workload to forecast workload for each microservice, and the predicting results are utilized by *Microservice Scaler* which leverages an efficient online algorithm to determine the instance number of each microservice jointly with making placement decisions.

The local control module is composed of a *Burst Handler*, and a *Load Balancer*. It runs inside each instance along with the microservice program. When an instance deployed on the edge device receives a request, the *Burst Handler* decides whether the request should be processed locally or be offloaded to the cloud. Moreover, *Load Balancer* selects the target instance for each microservice to be called according to the processing rate.

The timeslot length (time interval) for ProScale to make autoscaling decisions is set to one minute according to the previous works [26], [27], [21]. Since the time required for making predictions is negligible (§5.1), the time required for generating placement schema is no more than one second (§6.6), and the time required for deploying the new placement schema is about 350ms [41], the total time for ProScale to scale microservices at a time is about only one second, which is very short compared to the timeslot length, and hence the promptness of autoscaling won't be affected.

5.1 Predictor

Predictor forecasts workload for all the microservices in each timeslot. In order to ensure the timeliness of the predicting results, a fast predicting method must be employed. Therefore, we leverage the SMA method to predict the workload for each microservice.

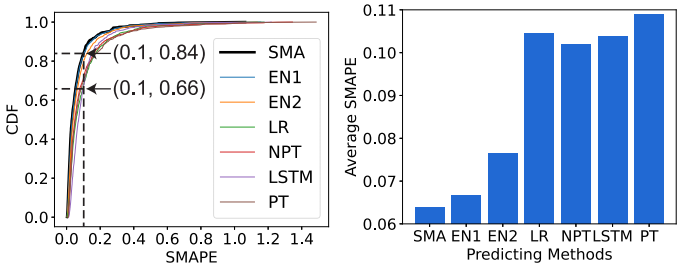


Fig. 8: Predicting results of different predicting methods. (a) Cumulative distribution of SMAPE. (b) Average SMAPE.

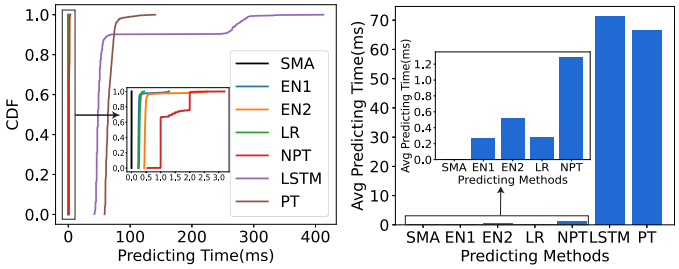


Fig. 9: The time required for different predicting methods to make a prediction. (a) Cumulative distribution of Predicting Time. (b) Average Predicting Time.

The SMA utilizes the mean of workload in the latest N timeslots as the predicting result for the next timeslot. The predicted workload of the tth timeslot for a given microservice can be calculated by

$$\hat{\psi}^t = \frac{\psi^{t-1} + \psi^{t-2} + ... + \psi^{t-N}}{N}, \tag{15}$$

where ψ^t represents the workload of the microservice in the tth timeslot. We use Symmetric Mean Absolute Percentage Error (SMAPE) to evaluate the accuracy of the prediction, which is calculated by

$$SMAPE = \frac{2}{n}\sum_{t=1}^{n}\frac{|\hat{\psi}^t - \psi^t|}{|\hat{\psi}^t| + |\psi^t|}. \tag{16}$$

A smaller SMAPE means a more accurate prediction.

We extract the workload sequences of 1000 randomly selected microservices from the Alibaba microservice trace and predict them using SMA with N=1 to N=10 respectively. As shown in Fig. 7a, the vast majority of predicting results have SMAPE less than 0.1, which indicates that most workload can be accurately predicted by SMA. Fig. 7b shows that when N=2, the SMA has the highest prediction accuracy. Therefore, SMA with N=2 is utilized by *Predictor*.

To further validate the effectiveness of SMA, we compare SMA with N=2 with two industrial forecasting tools open-sourced by Meta[53], Prophet (**PT**) [54] and NeuralProphet (**NPT**) [55], and four state-of-the-art predicting methods utilized by previous studies for predicting workload, Single Elastic Network Regression (**EN1**) [26], Double Elastic Network Regression (**EN2**) [27], Linear Regression (**LR**) [28], and Long Short Term Memory (**LSTM**) [29]. For PT and NPT, every 10 timeslots, the workload of the past 180 timeslots is used to train a model to predict the workload for the next 10 timeslots at once. For LSTM, every 10 timeslots, the workload of the past 180 timeslots is leveraged to train

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

a new LSTM model, and in each timeslot, the workload of the previous five timeslots is used as input to predict the workload for the timeslot.

We respectively apply PT, NPT, EN1, EN2, LR, LSTM, and SMA with N=2 to the previously extracted 1000 workload sequences. As shown in Fig. 8a, SMA with N=2 obtained SMAPE less than 0.1 on more workload sequences. Fig. 8b shows that the SMA with N=2 has the lowest average SMAPE, which represents the highest prediction accuracy. Furthermore, as shown in Fig. 9, the time required for other predicting methods to make a prediction is much longer than that of SMA, and a long predicting time hurts the timeliness of the predicting result. In a word, SMA outperforms all of these predicting methods in both accuracy and speed.

Predictor collects the workload in the last timeslot at the beginning of each timeslot, and then SMA with N=2 is utilized to forecast the workload for each microservice. After that, the predicting results will be sent to the *Microservice Scaler* to support generating the placement schema.

5.2 Microservice Scaler

In each time slot, after receiving the predicting results from *Predictor*, *Microservice Scaler* leverages them to generate the placement schema for reducing the maximal request response time. Then, it compares the newly generated placement schema $n_{m,d}^{new}$ with the last placement schema $n_{m,d}^{old}$. If $n_{m,d}^{new} < n_{m,d}^{old}$, it will terminate $n_{m,d}^{old} - n_{m,d}^{new}$ instances of microservice m on device d. If $n_{m,d}^{new} > n_{m,d}^{old}$, it will start $n_{m,d}^{new} - n_{m,d}^{old}$ new instances of microservice m on device d.

Due to the NP-hardness of the considered problem, it is impossible to find the optimal solution in polynomial time. Besides, since the placement decisions have to be made within a very short time to promptly respond to workload variation, we propose an efficient online algorithm to generate the placement schema.

5.2.1 Algorithm Procedure

According to Eq. (4), a greedy algorithm naturally comes up to scale microservices by minimizing the makespan for each instance. However, due to the limited resources of edge devices, there are always instances deployed on the cloud, which leads to inevitable backhaul transmission and hurts the request response time. Therefore, we specifically design a procedure for the algorithm to reduce the edge-cloud transmission.

As described in Algorithm 1, the MicroserviceScaling (MS) algorithm first initializes the placement schema by setting $n_{m,d}$ to 0 for each microservice and each device (lines 1-3), and then it places the instances of m_{start} to all the edge devices (lines 4-8). Next, the MS algorithm scales each microservice in the topological order. For each microservice, the MS algorithm deploys instances one by one for it until the workload demand is satisfied (lines 12-18). For each instance, the MS algorithm greedily chooses the device for deploying it using the GreedyDeviceChosen (GDC) algorithm described in Algorithm 2. The GDC algorithm traverses the devices and chose the device with minimal makespan for the instance to be deployed.

For the microservices whose instances are both deployed on edge devices and the cloud, the MS algorithm redeploys all their instances on the cloud (lines 19-23). This step not only won't affect the maximal response time but also helps save resources of edge devices. Finally, the MS algorithm collects the microservices that have instances deployed on the edge devices and all its UM instances are deployed on the cloud (lines 24-31), and utilize the BackhaulTransmissionReduction (BTR) algorithm described in Algorithm 3 to reduce the edge-cloud transmission as much as possible.

The underlying idea of the BTR algorithm is to reduce the backhaul transmission by redeploying the microservices with large downstream backhaul transmission and small upstream transmission on the cloud. For each collected microservice, the BTR algorithm calculates its maximal up-

Algorithm 1: MicroserviceScaling (MS)

Input: $\mathcal{M}$: microservice set of the application
$\hat{\psi}_m$: predicted workload for $\forall m \in \mathcal{M}$
Output: $\mathcal{N}$: the overall placement schema

```
// Initialization
1  for each m ∈ M do
2      for each d ∈ D do
3          n_{m,d} ⟵ 0
4  I ⟵ create an empty set
5  for each d ∈ E do
6      i ⟵ create an instance
7      x_i ⟵ m_start, y_i ⟵ d, L_i ⟵ 0
8      n_{m_start,d} ⟵ 1, I ⟵ I ∪ {i}
// Greedy instance placement
9  Sort M in topological order
10 for each m ∈ M\{m_start} do
11     I_m ⟵ create an empty set
12     while ψ̂_m > 0 do
13         U_m ⟵ obtain the UM set of m
14         d*, τ_min ⟵ GreedyDeviceChosen(m, U_m)
15         i ⟵ create an instance
16         x_i ⟵ m, y_i ⟵ d*, L_i ⟵ τ_min
17         I ⟵ I ∪ {i}, I_m ⟵ I_m ∪ {i}
18         n_{m,d*} ⟵ n_{m,d*} + 1, ψ̂_m ⟵ ψ̂_m − r_{m,d*}
// Edge-cloud microservice redeployment
19 for each m ∈ M\{m_start} do
20     if m has instances both on edge devices and cloud then
21         for each i ∈ I_m do
22             n_{m,y_i} ⟵ n_{m,y_i} − 1, n_{m,C} ⟵ n_{m,C} + 1
23             y_i ⟵ C
// Backhaul transmission reduction
24 M' ⟵ M\{m_start}
25 for each m ∈ M\{m_start} do
26     if m has instances on edge devices then
27         D_m ⟵ obtain the DM set of m
28         for each m' ∈ D_m do
29             if m' has instances on edge devices then
30                 M' ⟵ M\{m}
31                 break
32 BackhaulTransmissionReduction(M')
33 return N
```

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

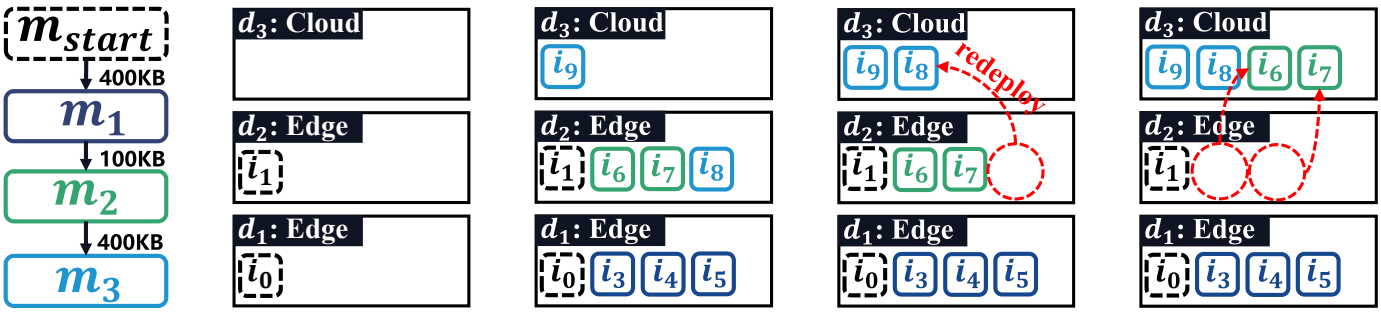


Fig. 10: Illustration of the execution example of the MS algorithm, where different colors represent different microservices. (a) The input application structure. (b) Initialization. (c) Greedy instance placement. (d) Edge-cloud microservice redeployment. (e) Backhaul transmission reduction.

Algorithm 2: GreedyDeviceChosen (GDC)

Input: m: the microservice to be placed
$\mathcal{U}_m$: the UM set of m
Output: d^*: the selected device
τ_{min}: the makespan of the instance of microservice m placed on d^*

$\tau_{min} \longleftarrow \infty$
for *each* $d \in \mathcal{D}$ **do**
  if d *has enough available reources to deploy* m **then**
    $\tau \longleftarrow 0$
    for *each* $m' \in \mathcal{U}_m$ **do**
      for *each* $i' \in \mathcal{I}_{m'}$ **do**
        $\tau' \longleftarrow L_{i'} + T^{Tran}_{y_{i',d},\Delta_{x_{i',m}}} + T^{Proc}_{m,d}$
        $\tau \longleftarrow \max(\tau, \tau')$
    if $\tau < \tau_{min}$ **then**
      $\tau_{min} \longleftarrow \tau$
      $d^* \longleftarrow d$
return d^*, τ_{min}

Algorithm 3: BackhaulTransmissionReduction (BTR)

Input: $\mathcal{M}'$: microservices with backhaul transmission
while $\mathcal{M}' \neq \emptyset$ **do**
  $m \longleftarrow$ obtain a microservice from $\mathcal{M}'$
  $\mathcal{M}' \longleftarrow \mathcal{M}' \backslash \{m\}$
  $\mathcal{U}_m, \mathcal{D}_m \longleftarrow$ obtain the UM set and DM set of m
  if $\mathcal{U}_m = \emptyset$ *or* $\mathcal{D}_m = \emptyset$ **then**
    continue
  $\Delta^{UM}_m \longleftarrow 0$
  for *each* $m' \in \mathcal{U}_m$ **do**
    $\Delta^{UM}_m \longleftarrow \max(\Delta^{UM}_m, \Delta_{m,m'})$
  $\Delta^{DM}_m \longleftarrow \infty$
  for *each* $m' \in \mathcal{D}_m$ **do**
    $\Delta^{DM}_m \longleftarrow \min(\Delta^{DM}_m, \Delta_{m,m'})$
  if $\Delta^{DM}_m > \Delta^{UM}_m$ **then**
    for *each* $i \in \mathcal{I}_m$ **do**
      $n_{m,y_i} \longleftarrow n_{m,y_i} - 1, n_{m,C} \longleftarrow n_{m,C} + 1$
      $y_i \longleftarrow C$
    for *each* $m' \in \mathcal{U}_m$ **do**
      if *all the DM of* m' *are deployed on cloud* **then**
        $\mathcal{M}' \longleftarrow \mathcal{M}' \cup \{m'\}$

stream data transmission and minimal downstream data transmission, and if the minimal downstream data transmission is bigger than the maximal upstream data transmission, all the instances of that microservice will be redeployed on the cloud. If a microservice is redeployed to the cloud, the BTR algorithm traverses its UM set and adds the UM whose DM instances are all deployed on the cloud to the collected microservices for the subsequent redeployment.

Although we only discuss the autoscaling of a single application, the MS algorithm can be easily extended to multiple applications. We can make autoscaling decisions for each application one by one, or introduce a virtual microservice and add edges between it and each entry microservice to make up a composite application, to which the MS algorithm can be applied.

5.2.2 Algorithm Execution Example

Fig.10 shows an example of scaling an application composed of three microservices in the edge computing environment using the MS algorithm. For the sake of simplicity, we make the following simplification in this example. (1) The three microservices of the application are serially connected into a chain and the data volume transmitted between m_{start} and m_1, m_1 and m_2, m_2 and m_3 is respectively 400KB, 100KB, and 400KB. (2) There are only two edge devices and a cloud data center in the edge computing environment. (3) All the microservices have the same resource requirements and each edge device can at most accommodate three instances while the cloud can deploy unlimited instances. (4) All the three devices have the same computing capability, which indicates that instances of the same microservice always have the same processing time and the same processing rate no matter where they are deployed. (5) The two edge devices have the same backhaul bandwidth and the same backhaul propagation latency.

According to the simplification (4), the instance number of microservice m can be directly calculated by $\lceil \frac{\hat{\psi}_m}{r_m} \rceil$ where $\hat{\psi}_m$ represents the predicted workload of m and r_m denotes the processing rate of m. In this example, m_1, m_2, and m_3 respectively require 3, 2, and 2 instances to meet the predicted workload demand.

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

As shown in Fig. 10b, the MS algorithm initializes the placement schema by deploying the instance of m_{start} to each edge device. Next, it scales each microservice and leverages the GDC algorithm to greedily decide the device to be deployed for each instance. As shown in Fig. 10c, after the greedy instance placement, the maximal request response time is the makespan of the instance i_9, which is equal to $l_{LAN} + \frac{400KB}{b_{LAN}} + T^{proc}_{m_1} + l_{LAN} + \frac{100KB}{b_{LAN}} + T^{proc}_{m_2} + l_{BHN} + \frac{400KB}{b_{BHN}} + T^{proc}_{m_3}$, where l_{LAN}, l_{BHN}, b_{LAN}, b_{BHN}, respectively represent the propagation latency and bandwidth of LAN and the backhaul network, and $T^{proc}_{m_1}$, $T^{proc}_{m_2}$, and $T^{proc}_{m_3}$ respectively represent the processing time of microservice m_1, m_2, and m_3. The request response time is obtained when a user request is received by the edge device d_2, and it is forwarded to the instance i_3 for processing, and then i_3 calls i_6 for subsequent processing, and finally i_6 calls i_9 to finish the request processing.

As shown in Fig. 10d, for the microservice m_3 whose instances are both deployed on the edge device and the cloud, the MS algorithm redeploys its instances on the cloud while the maximal request response time is not affected. As shown in Fig. 10e, the MS algorithm finally reduces the backhaul transmission with the BTR algorithm. As the instances of microservice m_2 are deployed on the edge devices and have backhaul transmission, the BTR algorithm checks the maximal UM transmission $\Delta^{UM}_{m_2}$ and minimal DM transmission $\Delta^{DM}_{m_2}$ of m_2 and find that $\Delta^{UM}_{m_2} = 100KB < \Delta^{DM}_{m_2} = 400KB$. Therefore the BTR algorithm redeploys the instances of m_2 on the cloud. When the MS algorithm ends, the maximal request response time is $l_{LAN} + \frac{400KB}{b_{LAN}} + T^{proc}_{m_1} + l_{BHN} + \frac{100KB}{b_{BHN}} + T^{proc}_{m_2} + T^{proc}_{m_3}$, where the large backhaul transmission between instances of m_2 and m_3 is eliminated.

5.2.3 Algorithm Complexity Analysis

Finally, we analyze the time complexity of the MS algorithm. We introduce

$$\lambda = \max_{m \in \mathcal{M}} \lceil \frac{\hat{\psi}_m}{\min_{d \in \mathcal{D}} r_{m,d}} \rceil \tag{17}$$

to represent the maximal number of instances that can be deployed of all the microservices, where $\hat{\psi}_m$ and $r_{m,d}$ respectively denote the predicted workload for m and the processing rate for the instance of m deployed on d. Besides, we introduce $\mathcal{G}$ to represent the set of call relationships between microservices. Hence,

$$\sum_{m \in \mathcal{M}} |\mathcal{U}_m| = |\mathcal{G}|, \sum_{m \in \mathcal{M}} |\mathcal{D}_m| = |\mathcal{G}|, \tag{18}$$

where $\mathcal{U}_m$ and $\mathcal{D}_m$ respectively denotes the UM set and DM set of the microservice m.

Therefore, the time complexity of the GDC algorithm is $O(|\mathcal{D}| \cdot |\mathcal{U}_m| \cdot \lambda)$. In addition, the time complexity of the BTR algorithm is $O(|\mathcal{M}| \cdot \lambda + |\mathcal{G}|)$ because the number of DM deployed on the edge device for each microservice can be recorded before the BTR algorithm starts, and hence the if-clause (line 18) can be implemented by updating the recorded number before the if-clause is executed and checking whether the recorded number is 0 with the complexity of $O(1)$. As the application structure is a connected DAG, $|\mathcal{M}| - 1 \leq |\mathcal{G}| \leq \frac{|\mathcal{M}| \cdot (|\mathcal{M}|-1)}{2}$, which indicates $|\mathcal{M}| = O(|\mathcal{G}|)$. As a result, the complexity of the MS algorithm is $O(\lambda^2 \cdot |\mathcal{D}| \cdot |\mathcal{G}|)$. Given the λ, the time complexity of the MS algorithm grows polynomially with the number of devices and the number of call relations between microservices.

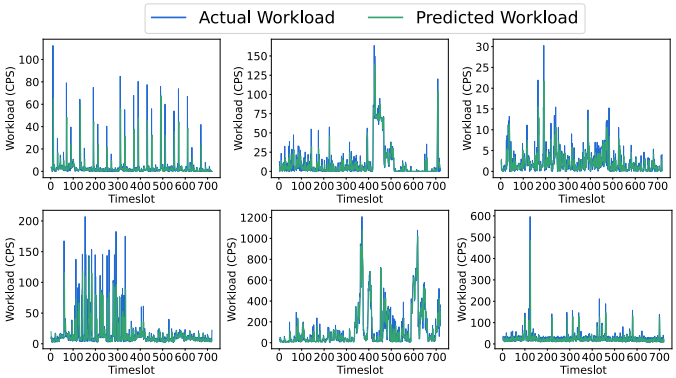


Fig. 11: The six workload sequences with the smallest prediction accuracy under SMA with N=2

5.3 Burst Handler

Burst Handler handles burst requests for the instances deployed on edge devices. As shown in Fig. 7a, although most workload sequences can be predicted accurately, there is still a fraction of sequences whose SMAPE is high. To find out the reason why these workload sequences are difficult to predict, we depict the six workload sequences with the highest SMAPE in Fig. 11. We can find that all of them have frequent bursts. Therefore, we suppose that the burst workload is difficult to be accurately predicted. To further verify our supposition, we confirm that the prediction error of burst workload is much more than that of non-burst workload by the following steps.

Given a workload sequence $\Psi =< \psi^1, \psi^2, ..., \psi^n >$ where ψ^t denotes the workload in tth timeslot, its variation in tth ($t > 1$) timeslot is calculated by

$$\nu^t = |\psi^t - \psi^{t-1}|. \tag{19}$$

Specifically, $v^1 = 0$. Since burst workload is the workload that has a very large variation, we identified the set of burst workload as

$$\mathcal{B} = \{\psi^t \in \Psi | \nu^t > \mu_\nu + 3 \cdot \sigma_\nu\}, \tag{20}$$

where μ_ν and σ_ν respectively represent the mean and the standard deviation of ν^t for all timeslots. In addition, we compute the prediction error in tth timeslot by

$$\epsilon^t = |\hat{\psi}^t - \psi^t|, \tag{21}$$

where $\hat{\psi}^t$ denotes the predicted workload of tth timeslot. Furthermore, the average prediction error of burst workload and non-burst workload can be respectively calculated with

$$e_b = \frac{\sum_{\psi^t \in \mathcal{B}} \epsilon^t}{|\mathcal{B}|} \tag{22}$$

and

$$e_{\neg b} = \frac{\sum_{\psi^t \in \Psi \wedge \psi^t \notin \mathcal{B}} \epsilon^t}{|\Psi| - |\mathcal{B}|}. \tag{23}$$

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

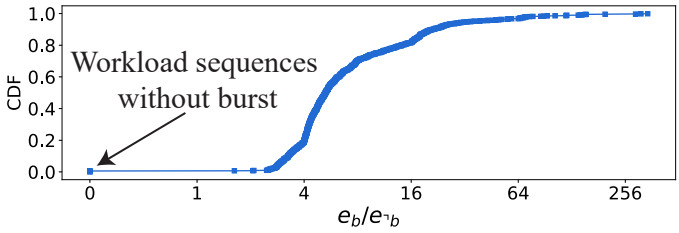


Fig. 12: Cumulative distribution of $e_b/e_{\neg b}$ of each workload sequence.

We calculate $\frac{e_b}{e_{\neg b}}$ of all the previously extracted 1000 workload sequences. As shown in Fig.12, except for the sequences without burst, all the other workload sequences have $\frac{e_b}{e_{\neg b}}$ greater than 1, and the majority of workload series has $\frac{e_b}{e_{\neg b}}$ greater than 4 which suggests that the prediction error of burst workload is much more than that of non-burst workload. Furthermore, there are even sequences with $\frac{e_b}{e_{\neg b}}$ over 256, which further verifies our supposition.

We also confirm that none of the forecasting methods mentioned in §5.1 can accurately predict bursts through experiments, the concrete confirmation is provided in the supplementary materials. Therefore, the burst handling mechanism must be proposed to complement the prediction-driven proactive microservice autoscaling.

Since bursts make the call rate exceed the processing rate for instances and cause lots of requests queued in the edge devices, thus hurting the request response time, *Burst Handler* sets the maximum queuing time for each instance deployed on edge devices to offload overloaded requests caused by bursts to the cloud data center in time. When a new request is received by an instance, *Burst Handler* calculates the queuing time of the received request by $T^{proc} \cdot q$ where T^{proc} denotes the processing time for the instance and q represents the queue length which denotes the number of requests waiting to be processed. If the queuing time exceeds the maximum limit, the request will be forwarded to the cloud for in-time processing. Besides, additional instances of each microservice are deployed on the cloud specifically for processing burst requests.

The key to *Burst Handler* is setting a suitable maximum queuing time for instance deployed on the edge device. If the maximum queuing time is set too long, requests will be queued on the edge device for a long time, thus hurting the request response time. If the maximum queuing time is set too short, most requests will be directly forwarded to the cloud, and hence not only leads to a long backhaul transmission time but also causes a low resource utilization of the edge device. Therefore, to make a tradeoff between the request queuing time, backhaul transmission time, and resource utilization of the edge device, we set the maximum queuing time to 4 times the time required to transfer the request from the deployed edge device to the cloud for the instances deployed on edge devices.

5.4 Load Balancer

Load Balancer leverages the LoadBalance (LB) algorithm described in Algorithm 4 to balance the load, hence avoiding requests piling up on instances deployed on weak devices while taking full advantage of the computing capability of instances deployed on powerful devices.

The design idea of *Load Balancer* is to match the call rate of each instance with its processing rate, which makes the resources of each instance neither wasted nor insufficient, thus achieving maximum utilization of edge resources. As described in Algorithm 4, when an instance decides to send a request, *Load Balancer* will select an instance of the dest microservice for it. To make full use of edge devices, if the source instance is deployed on edge devices, the request will also be sent to an instance deployed on the edge device. However, if all the instances of the dest microservice are deployed on the cloud, the request can only be sent to the cloud (lines 1-4). Next, the processing rate is utilized as the weight for selecting the target instance to be called.

Algorithm 4: LoadBalance (LB)

Input: m: target microservice
d: device where the source instance is deployed
`// get candidate instances of` m
1 **if** $d \in \mathcal{E}$ **then**
2 $\quad$ $candidates \longleftarrow$ get the instances of m that are deployed on edge devices
3 **if** $d = C$ *or* $candidates$ *is empty* **then**
4 $\quad$ $candidates \longleftarrow$ get the instances of m that are deployed on the cloud
5 $\iota \longleftarrow$ randomly select an instance from the candidates with the processing rate as the weight of probability
6 send the request to ι

6 Experimental Evaluation

In this section, we first visually present the superiority of prediction-driven proactive autoscaling. Then the performance of ProScale in response time and resource consumption is respectively evaluated by trace-driven simulations. Moreover, we investigate the role of each component of ProScale in reducing response time under different workload levels. Furthermore, the execution time of making autoscaling decisions for ProScale and other baselines is also compared to validate the high efficiency of ProScale.

6.1 Experiment Setup and Compared Baselines

All the experiments are implemented in Python 3.9 and conducted on windows OS equipped with the 2.7 GHz Six-Core Intel Core i5-11500 and 16 GB RAM. We implement a simulator with more than 3000 lines of code to simulate the queuing and parallel processing procedure of requests.

Fig. 13 shows the three microservice-based applications used in the following experiments. The large-sized artificial application (ART) with 30 microservices is generated with the generator proposed in [14]. The small-sized smart surveillance application (PTZ) with only 4 microservices and the medium-sized video processing application (VP) with 12 microservices are respectively from [56] and [57]. Moreover, Table 2 lists the parameters used in the experiment. Latency and bandwidth of the network represent novel network technologies like 5G [58] for the backhaul network and Gigabit Ethernet [59] for LAN connections. The other parameters such as the processing time and resource capacity of devices are acquired from Alibaba cluster

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

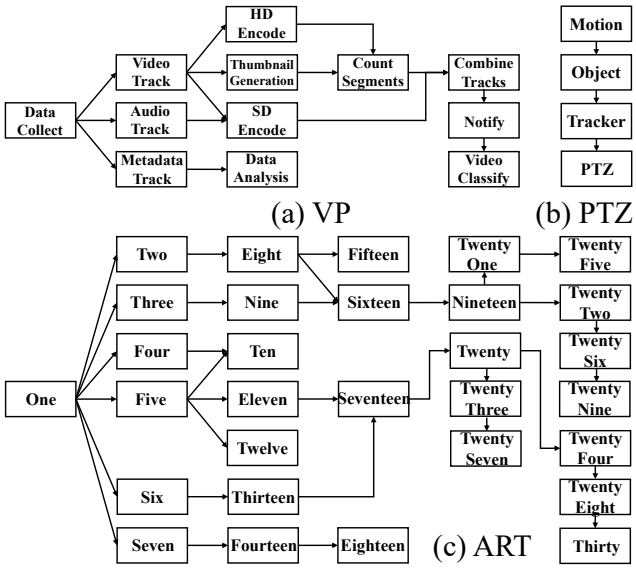


Fig. 13: Applications. (a) Medium-sized video processing application (VP) with 12 microservices. (b) Small-sized smart surveillance application (PTZ) with only 4 microservices. (c) Large-sized artificial application (ART).

trace[60] and previous works[11], [10]. In addition, the CPU core number, memory size, and storage volume of the cloud data center are set to infinite. Furthermore, the number of user requests per timeslot is evenly distributed to each edge device.

ProScale is compared with three strategies to microservice placement at the edge. The Cloud-Only is a designed baseline while QMPSO and SAA-RP are existing solutions that both employed meta-heuristic algorithms to place multiple instances for each microservice at the edge. In the following experiments, the parameters of meta-heuristic algorithms leveraged by QMPSO and SAA-RP are shown in Table 3. The competitors are described as follows.

1) **Cloud-Only**: All the microservice instances are deployed on the cloud and each microservice is overprovisioned with a large number of instances.
2) **QoS-aware Multi-objective Set-based Particle Swarm Optimisation (QMPSO)**: The static placement strategy proposed in [11] which assumes that each microservice has a constant workload. We adjust its optimization objective to request response time, and the constant workload of each microservice is set to the mean value of the corresponding workload sequence selected for each microservice.
3) **Sample Average Approximation-based Redundancy Placement (SAA-RP)**: The static placement strategy designed in [10] aims to minimize the request response time. It tries to place as many instances on edge devices as possible and randomly determines the instance number of each microservice for each device without considering the time-varying workload. Besides, each instance calls the closest instance of the dest DM to minimize the transmission time.

TABLE 2: Experiment Parameters

Parameter	Value	Parameter	Value
$b_{d,d'}$ (LAN)	1± 0.2 Gbps	Φ_d	128-1024 GB
$l_{d,d'}$ (LAN)	0.5 ms	γ_m	1-4
$b_{d,d'}$ (Backhaul)	150± 20 Mbps	θ_m	0.5-4 GB
$l_{d,d'}$ (Backhaul)	2 ms	ϕ_m	0.5-8 GB
Γ_d	8-96	$\Delta_{m,m'}$	100-3000 KB
Θ_d	16-64 GB	$T^{proc}_{m,d}$	10-50 ms

TABLE 3: Parameters for QMPSO and SAA-RP

QMPSO Parameter	Value	SAA-RP Parameter	Value
swarm size	50	population size	10
max iterations	300	max iterations	300
ω	0.4-0.9	p_c	0.8
c	1.49445	p_m	0.1

6.2 Prediction-driven Proactive Autoscaling

To visually exhibit the performance of the prediction-driven proactive autoscaling, we randomly selected six microservices and draw their workload sequences. Moreover, the corresponding predicting results and the number of instances over time are also depicted in each subgraph of Fig. 16, where the left and right vertical axis respectively represent the workload and the number of instances. As shown in Fig. 16, we can find that *Predictor* is capable to make accurate predictions thereby ProScale can proactively adjust the instance number for each microservice to accommodate the time-varying workload.

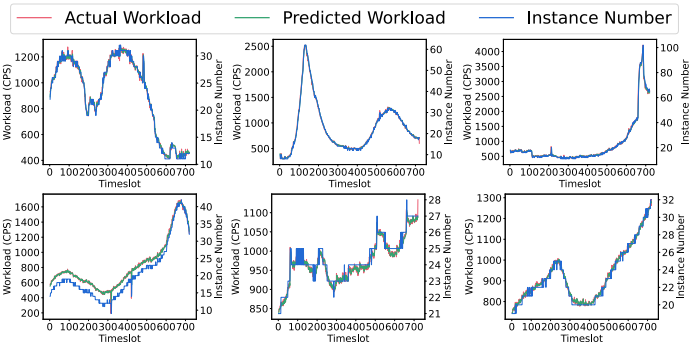


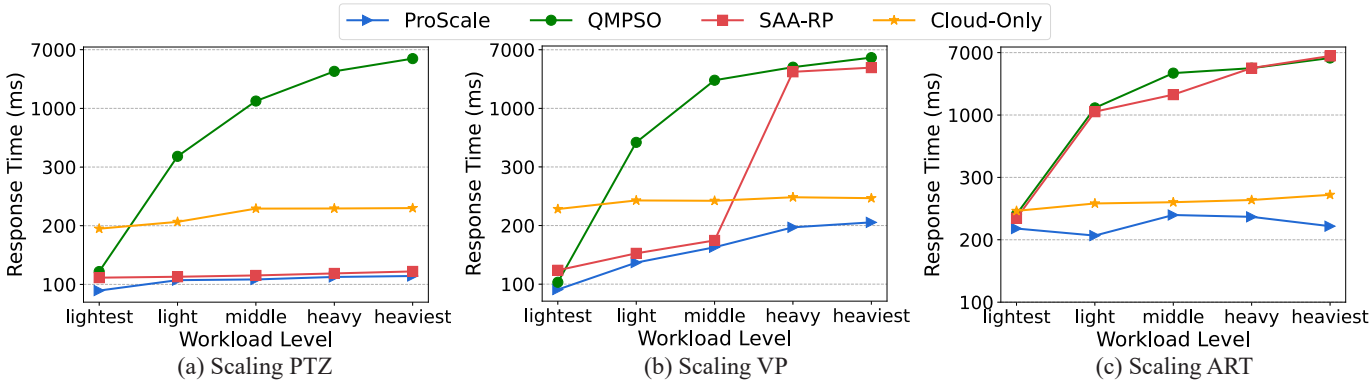
Fig. 16: Actual workload, predicted workload, and instance number over time for six randomly selected microservices.

6.3 Request Response Time

We divide all the workload sequences into five levels: lightest, light, middle, heavy, and heaviest, where the mean workload was respectively at [100, 200), [200, 400), [400, 600), [600, 800), and [800, 1000]. The workload level of the application is defined as the workload level of all the microservices. For workload levels from lightest to heaviest, we respectively deploy 2, 4, 6, 8, and 10 additional instances on the cloud for each microservice to specifically process burst requests.

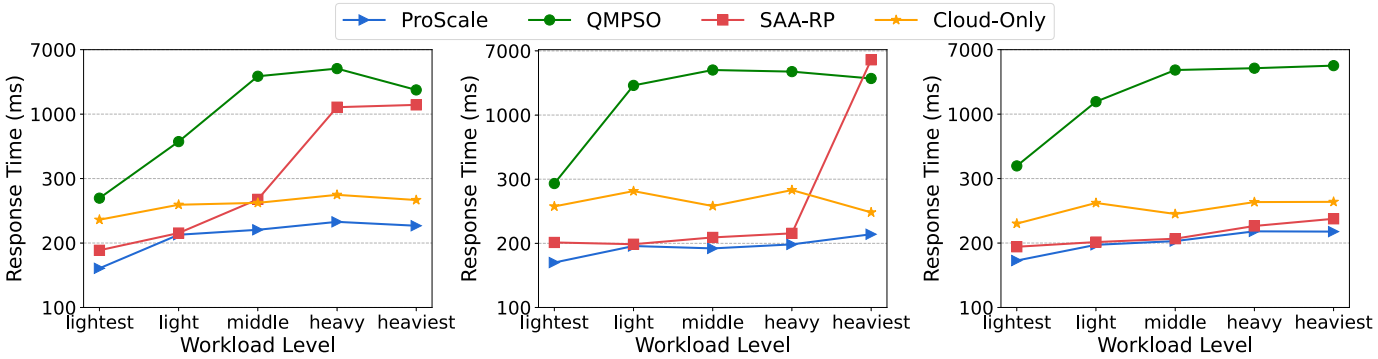
We first set the number of edge devices to 25 and respectively scale the three applications depicted in 13 with ProScale and other competitors to validate the performance of ProScale in reducing response time under the situation that the edge resources are limited. As shown in Fig. 14, we can find that ProScale outperforms all the other competitors and reduces average response time by 96.7% while

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429



(a) Scaling PTZ (b) Scaling VP (c) Scaling ART

Fig. 14: Average response time for scaling different applications at various workload levels when there are 25 edge devices.



(a) 50 Edge Devices (b) 75 Edge Devices (c) 100 Edge Devices

Fig. 15: Average response time for scaling the ART application at various workload levels under different number of edge devices.

scaling the ART application at the heaviest workload level compared with SAA-RP.

We then confirm the scalability of ProScale by scaling the large-sized ART application under different number of edge devices with different strategies. As shown in 15, ProScale performs best and reduces average response time by 96.1% when the number of edge devices is 100 and the workload level is heaviest compared with QMPSO.

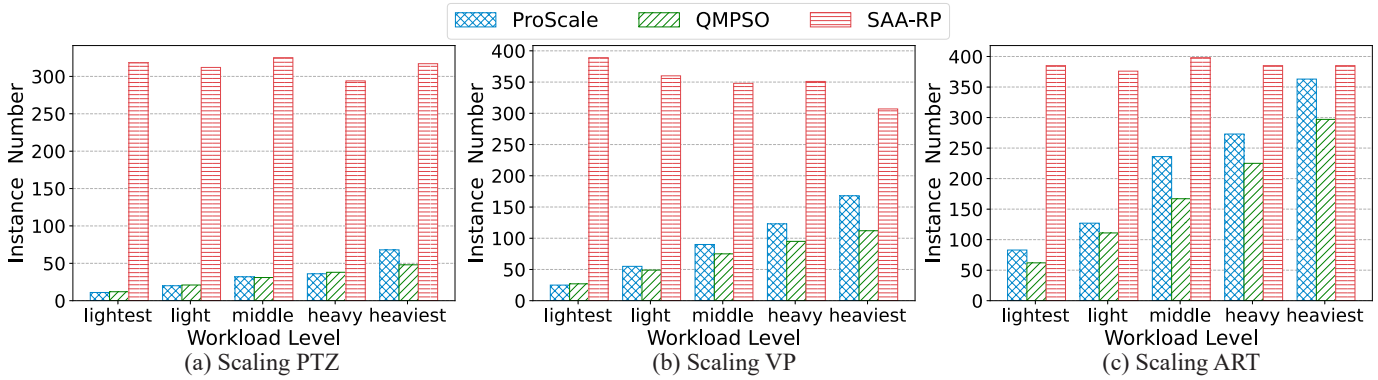
Since *Predictor* can accurately predict the workload, ProScale can adjust the instance number of each microservice to accommodate the time-varying workload. Besides, *Microservice Scaler* can leverage the predicting results to determine the number of instances for each microservice and then place them on edge devices to provide on-site processing, which reduces the amount of data transmitted through the backhaul network thus reducing the request response time. When burst requests occur, *Burst Handler* offloads the overloaded requests to the cloud for processing in time to prevent requests from piling up in the queue of instances deployed on edge devices. *Load Balancer* sends more requests to the instances with a high process rate, which takes full advantage of the computing capability of edge devices. In summary, ProScale has such an excellent performance due to all its four components working well.

Cloud-Only deploys a large number of instances for each microservice on the cloud data center to provide sufficient processing capability without considering deploying microservice instances on edge devices. Therefore, it can not leverage edge devices to reduce the response time. Although it can handle the time-varying workload with overprovision, it does not perform as well as ProScale.

QMPSO supposes that the workload of each microservice is fixed all the time and accordingly employs the static placement strategy. As a result, if the actual workload exceeds the supposed workload, there will be requests waiting in the queue, hence negatively affecting performance. Moreover, when burst requests occur, there will be a large number of queued requests, thus further increasing the request response time. We can also find that the average request response time of QMPSO becomes longer as the workload level becomes higher from Fig. 14 and Fig. 15. That's because the higher the workload level is, the more queued requests are, thereby resulting in worse performance.

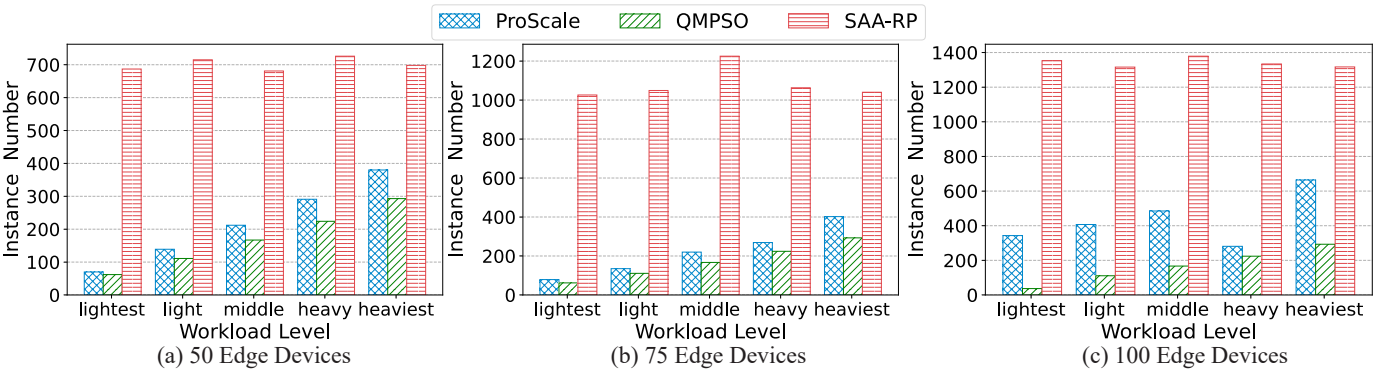
In SAA-RP, each instance only calls the DM instance with the minimum transmission time. Since for each instance deployed on the edge device, the transmission time to edge devices is much shorter than that to the cloud, the vast majority of calls are handled by instances deployed on edge devices. However, because SAA-RP randomly determines the instance number of each microservice for each device, if the number of edge devices is constrained or the application to be scaled is large-sized, there will be some microservices whose number of instances is too small to meet the workload demand. Therefore, a large number of requests will pile up in the instances of these microservices thus resulting in a long queuing time. As shown in Fig. 14c, when the number of edge devices is only 25, the performance of SAA-RP is tens of times worse than ProScale and Cloud-Only when the workload level is high. However, as shown in Fig. 14a and Fig. 15c, when the application is small-sized or the number of edge devices is large, SAA-RP outperforms Cloud-Only and QMPSO because a great many instances of each mi-

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429



(a) Scaling PTZ (b) Scaling VP (c) Scaling ART

Fig. 17: Number of instances deployed on edge devices for scaling different applications at various workload levels when there are 25 edge devices.



(a) 50 Edge Devices (b) 75 Edge Devices (c) 100 Edge Devices

Fig. 18: Number of instances deployed on edge devices for scaling different applications at various workload levels under different number of edge devices while scaling the ART application.

croservice are placed on edge devices to provide sufficient processing capability. Nevertheless, SAA-RP performs well only when the application is small-sized or the number of edge devices is adequate while ProScale performs well under all situations.

6.4 Resource Consumption

We compare the performance of the four strategies in terms of resource consumption which is denoted as the number of instances deployed on edge devices. Since ProScale adjusts the instance number for each microservice in every timeslot, we use the average number of instances deployed on edge devices in all timeslots to represent the resource consumption for ProScale.

As shown in Fig. 17 and Fig. 18, ProScale can reduce resource consumption by 5.7% to 96.5% compared with SAA-RP because SAA-RP places as many instances on edge devices as possible. QMPSO places fewer instances on edge devices than ProScale in most cases since it assumes the workload of each microservice does not change over time and employs the static placement strategy. Hence, when the workload exceeds the assumed fixed workload, QMPSO does not deploy new instances for adapting. Even though ProScale deploys more instances on edge devices than QMPSO in most cases, it can handle workload variation and burst requests, thus greatly outperforming QMPSO in terms of average response time.

6.5 Contribution of Each Component

We investigate the contribution of each component of ProScale to performance improvement in this section. We combine the four components of ProScale one after another and compare their performance in terms of average response time at different workload levels.

We first apply *Predictor* to the cloud and name this strategy as **Cloud+Predictor**. In each timeslot, the instance number of microservice m is calculated through $\lceil \frac{\hat{\psi}_m}{r_{m,C}} \rceil$ where $\hat{\psi}_m$ represents the predicted workload of m in the timeslot and $r_{m,C}$ denotes the processing rate of the instance of m deployed on the cloud. Next, *Microservice Scaler* is added to Cloud+Predictor and the new strategy is named **+Scaler**. In +Scaler, microservice instances call instances of the target microservice in a random manner with equal probability. Subsequently, we combine +Scaler with *Load Balancer* and name the combined strategy as **+LoadBalancer**. Finally, the *Burst Handler* is added to +LoadBalancer for offloading overloaded requests to the cloud, and all the components of ProScale are integrated. For the convenience of demonstrating the contribution of each component, we refer to ProScale as **+BurstHandler** in this section.

From Fig. 19, we can find that each component plays a different role under different workload levels, and it is their collaboration that makes ProScale has such excellent performance. *Microervice Scaler* plays the most important role when the workload level is lightest and light because it deploys microservice instances on edge devices to provide on-site processing and hence reduces the response

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

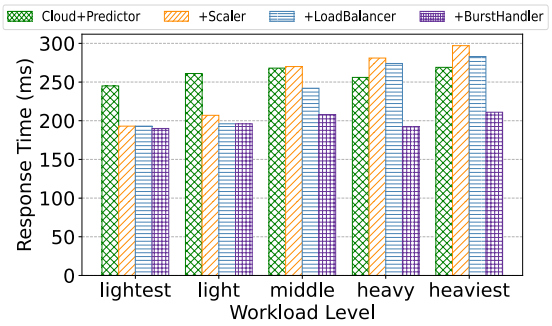


Fig. 19: Performance contribution of each component at different workload levels when there are 25 edge devices.

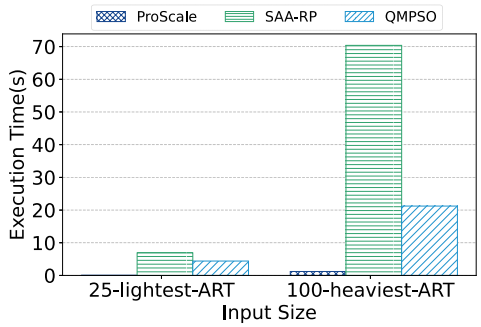


Fig. 20: Execution time under different input size

time. However, when the workload level becomes higher, *Microervice Scaler* leads to increased request response time. There are two main reasons for this issue. One reason is that the processing rate of each instance does not match its call rate because of the randomized load balancing mechanism, which leads to requests queuing on the instances with a low processing rate, especially when the call rate is high. The other reason is that the burst request volume is large when the workload level is high, which further increases the number of requests piled up on edge devices and causes a long response time.

LoadBalancer balances the load according to the processing rate of instances and *BurstHandler* offloads overloaded requests to the cloud, which resolves the above-mentioned two problems as shown in Fig. 19. With the four components working together, ProScale achieves excellent performance in reducing request response time by taking advantage of the on-site processing of edge devices while avoiding the problems brought by the heterogeneous and limited edge resources.

6.6 Execution Time

Fig. 20 compares the execution time of generating the placement schema for the ART application as the input size grows. The 25-lightest represents that the number of edge devices is 25 and the workload level of the application is the lightest, and the 100-heaviest can be explained in the same way. Since the competitors both leverage the static placement strategy, while ProScale makes placement decisions in each time slot, for ProScale, the execution time is the mean value of that in all the timeslots. As shown in Fig. 20, the execution time of the two competitors under different input sizes is considerably higher than ProScale, which verifies the efficiency of the MS algorithm employed by the *Microservice Scaler* of ProScale.

7 Conclusion

This paper proposes ProScale to achieve proactive autoscaling in the edge computing environment. ProScale is composed of a predictor, a microservice scaler, a burst handler, and a load balancer. All the four components are designed to solve the microservice scaling problem of each time slot. The predictor forecasts the workload for all the microservices at the beginning of each timeslot so that the scaler can leverage the predicted workload to replace the actual workload to resolve the problem of microservice scaling at the edge and generate the placement schema. Moreover, the load balancer matches the call rates of instances with their processing rates, making the edge resources allocated by the scaler to be fully utilized. Furthermore, the burst handler is designed as a complementation of ProScale, which handles unpredictable bursts with the edge-cloud collaboration.

Through extensive trace-driven experiments, we find that ProScale outperforms all the competitors in reducing response time while saving edge resources. Moreover, ProScale has great scalability. Furthermore, all the four components are validated to make contributions to performance improvement.

The main limitation of this work is that it is only based on the trace-driven simulation and ProScale is not implemented on a real system. We plan to implement ProScale on the top of Kubernetes in the future. In the implementation, we will not only consider adjusting the number of instances for each microservice but also dynamically adjusting the resource allocation of each instance, so as to further reduce the request response time and improve the resource efficiency. At the same time, we will take the resource contention between instances into account while making placement decisions.

Acknowledgement

We thank the editor and anonymous reviewers. This work was supported in part by NSFC (61872175, 61832008), the Fundamental Research Funds for the Central Universities (2022300297), and the Collaborative Innovation Center of Novel Software Technology and Industrialization.

References

[1] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 3–18.

[2] Z. Ding, S. Wang, and C. Jiang, "Kubernetes-oriented microservice placement with dynamic resource allocation," *IEEE Transactions on Cloud Computing*, no. 01, pp. 1–1, 2022.

[3] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: yesterday, today, and tomorrow," *Present and ulterior software engineering*, pp. 195–216, 2017.

[4] Google, 2022. [Online]. Available: https://www.google.com/

[5] Amazon web service, 2022. [Online]. Available: https://aws.amazon.com/

[6] Netflix, 2022. [Online]. Available: https://www.netflix.com/

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

[7] Alibaba, 2022. [Online]. Available: https://www.alibabagroup.com/

[8] Hybrid cloud with aws, 2022. [Online]. Available: https://aws.amazon.com/hybrid/

[9] Building enterprise edge applications with google cloud. [Online]. Available: https://cloud.google.com/blog/topics/hybrid-cloud/google-cloud-tools-to-manage-your-enterprise-edge-apps

[10] H. Zhao, S. Deng, Z. Liu, J. Yin, and S. Dustdar, "Distributed redundancy scheduling for microservice-based applications at the edge," *IEEE Transactions on Services Computing*, 2020.

[11] S. Pallewatta, V. Kostakos, and R. Buyya, "Qos-aware placement of microservices-based iot applications in fog computing environments," *Future Generation Computer Systems*, vol. 131, pp. 121–136, 2022.

[12] Y. Li, H. Zhang, W. Tian, and H. Ma, "Joint optimization of autoscaling and adaptive service placement in edge computing," in *2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2021, pp. 923–930.

[13] X. He, Z. Tu, M. Wagner, X. Xu, and Z. Wang, "Online deployment algorithms for microservice systems with complex dependencies," *IEEE Transactions on Cloud Computing*, 2022.

[14] S. Luo, H. Xu, C. Lu, K. Ye, G. Xu, L. Zhang, Y. Ding, J. He, and C. Xu, "Characterizing microservice dependency and performance: Alibaba trace analysis," in *Proceedings of the ACM Symposium on Cloud Computing (SOCC)*, 2021, pp. 412–426.

[15] Z. Ding and Q. Huang, "Copa: A combined autoscaling method for kubernetes," in *2021 IEEE International Conference on Web Services (ICWS)*. IEEE, 2021, pp. 416–425.

[16] M. R. Hossen, M. A. Islam, and K. Ahmed, "Practical efficient microservice autoscaling with qos assurance," in *Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing*, 2022, p. 240252.

[17] F. Zhang, X. Tang, X. Li, S. U. Khan, and Z. Li, "Quantifying cloud elasticity with container-based autoscaling," *Future Generation Computer Systems*, vol. 98, pp. 672–681, 2019.

[18] A. Kwan, J. Wong, H.-A. Jacobsen, and V. Muthusamy, "Hyscale: Hybrid and network scaling of dockerized microservices in cloud data centres," in *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2019, pp. 80–90.

[19] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "Firm: An intelligent fine-grained resource management framework for slo-oriented microservices," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020, pp. 805–825.

[20] G. Yu, P. Chen, and Z. Zheng, "Microscaler: Cost-effective scaling for microservice applications in the cloud with an online learning approach," *IEEE Transactions on Cloud Computing*, 2020.

[21] A. U. Gias, G. Casale, and M. Woodside, "Atom: Model-driven autoscaling for microservices," in *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2019, pp. 1994–2004.

[22] B. Choi, J. Park, C. Lee, and D. Han, "phpa: A proactive autoscaling framework for microservice chain," in *5th Asia-Pacific Workshop on Networking (APNet)*, 2021, pp. 65–71.

[23] H. Yang, Q. Chen, M. Riaz, Z. Luan, L. Tang, and J. Mars, "Powerchief: Intelligent power allocation for multi-stage applications to improve responsiveness on power constrained cmp," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 133–146.

[24] Z. Yang, P. Nguyen, H. Jin, and K. Nahrstedt, "Miras: Model-based reinforcement learning for microservice resource allocation over scientific workflows," in *2019 IEEE 39th international conference on distributed computing systems (ICDCS)*. IEEE, 2019, pp. 122–132.

[25] M. Gotin, F. Lösch, R. Heinrich, and R. Reussner, "Investigating performance metrics for scaling microservices in cloudiot-environments," in *Proceedings of the 2018 ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2018, pp. 157–167.

[26] M. Abdullah, W. Iqbal, J. L. Berral, J. Polo, and D. Carrera, "Burst-aware predictive autoscaling for containerized microservices," *IEEE Transactions on Services Computing*, 2020.

[27] M. Abdullah, W. Iqbal, A. Mahmood, F. Bukhari, and A. Erradi, "Predictive autoscaling of microservices hosted in fog microdata center," *IEEE Systems Journal*, vol. 15, no. 1, pp. 1275–1286, 2020.

[28] W. Iqbal, A. Erradi, M. Abdullah, and A. Mahmood, "Predictive auto-scaling of multi-tier applications using performance varying cloud resources," *IEEE Transactions on Cloud Computing*, 2019.

[29] I. Prachitmutita, W. Aittinonmongkol, N. Pojjanasuksakul, M. Supattatham, and P. Padungweang, "Auto-scaling microservices on iaas under sla with cost-effective framework," in *2018 Tenth International Conference on Advanced Computational Intelligence (ICACI)*. IEEE, 2018, pp. 583–588.

[30] W. Iqbal, M. N. Dailey, and D. Carrera, "Unsupervised learning of dynamic resource provisioning policies for cloud-hosted multitier web applications," *IEEE Systems Journal*, vol. 10, no. 4, pp. 1435–1446, 2015.

[31] L. Liu, H. Tan, S. H.-C. Jiang, Z. Han, X.-Y. Li, and H. Huang, "Dependent task placement and scheduling with function configuration in edge computing," in *2019 IEEE/ACM 27th International Symposium on Quality of Service (IWQoS)*. IEEE, 2019, pp. 1–10.

[32] S. Y. Jang, B. Kostadinov, and D. Lee, "Microservice-based edge device architecture for video analytics," in *2021 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE Computer Society, 2021, pp. 165–177.

[33] Y. Wang, C. Zhao, S. Yang, X. Ren, L. Wang, P. Zhao, and X. Yang, "Mpcsm: Microservice placement for edge-cloud collaborative smart manufacturing," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 9, pp. 5898–5908, 2020.

[34] J. Deng, B. Li, J. Wang, and Y. Zhao, "Microservice pre-deployment based on mobility prediction and service composition in edge," in *2021 IEEE International Conference on Web Services (ICWS)*. IEEE, 2021, pp. 569–578.

[35] K. Ray, A. Banerjee, and N. C. Narendra, "Proactive microservice placement and migration for mobile edge computing," in *2020 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 2020, pp. 28–41.

[36] L. Chen, Y. Xu, Z. Lu, J. Wu, K. Gai, P. C. Hung, and M. Qiu, "Iot microservice deployment in edge-cloud hybrid environment using reinforcement learning," *IEEE Internet of Things Journal*, vol. 8, no. 16, pp. 12 610–12 622, 2020.

[37] L. Gu, D. Zeng, J. Hu, B. Li, and H. Jin, "Layer aware microservice placement and request scheduling at the edge," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 2021, pp. 1–9.

[38] L. Gu, D. Zeng, J. Hu, H. Jin, S. Guo, and A. Y. Zomaya, "Exploring layered container structure for cost efficient microservice deployment," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 2021, pp. 1–9.

[39] S. Deng, Z. Xiang, J. Taheri, M. A. Khoshkholghi, J. Yin, A. Y. Zomaya, and S. Dustdar, "Optimal application deployment in resource constrained distributed edges," *IEEE Transactions on Mobile Computing*, vol. 20, no. 5, pp. 1907–1923, 2020.

[40] W. Lv, Q. Wang, P. Yang, Y. Ding, B. Yi, Z. Wang, and C. Lin, "Microservice deployment in edge computing based on deep q learning," *IEEE Transactions on Parallel and Distributed Systems*, 2022.

[41] K. Fu, W. Zhang, Q. Chen, D. Zeng, and M. Guo, "Adaptive resource efficient microservice deployment in cloud-edge continuum," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 8, pp. 1825–1840, 2021.

[42] D. Taibi and V. Lenarduzzi, "On the definition of microservice bad smells," *IEEE software*, vol. 35, no. 3, pp. 56–62, 2018.

[43] H. Zhou, M. Chen, Q. Lin, Y. Wang, X. She, S. Liu, R. Gu, B. C. Ooi, and J. Yang, "Overload control for scaling wechat microservices," in *Proceedings of the ACM Symposium on Cloud Computing (SOCC)*, 2018, pp. 149–161.

[44] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE internet of things journal*, vol. 3, no. 5, pp. 637–646, 2016.

[45] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE communications surveys & tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.

[46] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of computer computations*. Springer, 1972, pp. 85–103.

[47] M. Du, Y. Wang, K. Ye, and C. Xu, "Algorithmics of cost-driven computation offloading in the edge-cloud environment," *IEEE Transactions on Computers*, vol. 69, no. 10, pp. 1519–1532, 2020.

[48] Y.-H. Kao, B. Krishnamachari, M.-R. Ra, and F. Bai, "Hermes: Latency optimal task assignment for resource-constrained mobile computing," *IEEE Transactions on Mobile Computing*, vol. 16, no. 11, pp. 3056–3069, 2017.

[49] S. Sundar and B. Liang, "Offloading tasks with communication

This article has been accepted for publication in IEEE Transactions on Parallel and Distributed Systems. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TPDS.2023.3238429

delay and deadline constraint," in *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 2018, pp. 37–45.
[50] W. Bao, D. Yuan, B. B. Zhou, and A. Y. Zomaya, "Prune and plant: Efficient placement and parallelism of virtual network functions," *IEEE Transactions on Computers*, vol. 69, no. 6, pp. 800–811, 2020.
[51] X. Zhang, Z. Qian, S. Zhang, and S. Lu, "Pattern-aware reliable virtual network function chain deployment," in *2017 IEEE International Symposium on Parallel and Distributed Processing with Applications and 2017 IEEE International Conference on Ubiquitous Computing and Communications (ISPA/IUCC)*. IEEE, 2017, pp. 335–342.
[52] Y. Xie, S. Wang, and Y. Dai, "Provable algorithm for virtualised network function chain placement in dynamic environment," in *2019 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2019, pp. 1–6.
[53] Meta, 2022. [Online]. Available: https://about.facebook.com/
[54] Prophet, 2018. [Online]. Available: https://github.com/facebook/prophet
[55] Neuralprophet, 2021. [Online]. Available: https://github.com/ourownstory/neural_prophet
[56] K. Hong, D. Lillethun, U. Ramachandran, B. Ottenwälder, and B. Koldehofe, "Mobile fog: A programming model for large-scale applications on the internet of things," in *Proceedings of the second ACM SIGCOMM workshop on Mobile cloud computing*, 2013, pp. 15–20.
[57] Q. Huang, P. Ang, P. Knowles, T. Nykiel, I. Tverdokhlib, A. Yajurvedi, P. Dapolito IV, X. Yan, M. Bykov, C. Liang *et al.*, "Sve: Distributed video processing at facebook scale," in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017, pp. 87–103.
[58] D. Minovski, N. Ogren, C. Ahlund, and K. Mitra, "Throughput prediction using machine learning in lte and 5g networks," *IEEE Transactions on Mobile Computing*, 2021.
[59] A. Yousefpour, A. Patil, G. Ishigaki, I. Kim, X. Wang, H. C. Cankaya, Q. Zhang, W. Xie, and J. P. Jue, "Fogplan: A lightweight qos-aware dynamic fog service provisioning framework," *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 5080–5096, 2019.
[60] Alibaba cluster trace, 2018. [Online]. Available: https://github.com/alibaba/clusterdata/tree/v2018



Ke Cheng received his bachelor's degree in Information Management and Information System with a minor degree in Computer science and Technology from Nanjing University, Nanjing, China, in 2021. He is currently working toward a PhD degree in the Department of Computer Science and Technology, Nanjing University. He is a member of the State Key Laboratory for Novel Software Technology. His research interests include edge computing and blockchain.



Sheng Zhang is an associate professor in the Department of Computer Science and Technology, Nanjing University. He is also a member of the State Key Lab. for Novel Software Technology. He received the BS and PhD degrees from Nanjing University in 2008 and 2014, respectively. His research interests include cloud computing and edge computing. To date, he has published more than 80 papers, including those appeared in *JSAC*, *TMC*, *TON*, *TPDS*, *TC*, *MobiHoc*, *ICDCS*, and *INFOCOM*. He received the Best Paper Award of IEEE ICCCN 2020 and the Best Paper Runner-Up Award of IEEE MASS 2012. He is the recipient of the 2020 ACM Nanjing Rising Star Award and the 2015 ACM China Doctoral Dissertation Nomination Award. He is a member of ACM, and a senior member of IEEE, CCF.



Chenghong Tu is a postgraduate student in the Department of Computer Science and Technology, Nanjing University. He received the BS degree from Nanjing Institute of Technology in 2020. His research interests include distributed computing, edge computing, heterogeneous computing and stream processing.



Xiaohang Shi received the BS degree from the Department of Computer Science and Technology, Nanjing University, Nanjing, China, in 2020, where he is currently working towards the PhD degree under the supervision of associate professor Sheng Zhang. He is a member of the State Key Laboratory for Novel Software Technology. Currently, his research interests include edge computing and video analytics.



Zhaoheng Yin received a B.Sc. degree from the department of computer science and technology at Nanjing University. He is working towards an M.Phil. degree at the Hong Kong University of Science and Technology (HKUST). He is a member of the Robotics Institute of HKUST. His research interests are machine learning, artificial intelligence, and robotics.



Sanglu Lu received her BS, MS, and PhD degrees from Nanjing University in 1992, 1995, and 1997, respectively, all in computer science. She is currently a professor in the Department of Computer Science and Technology and the State Key Laboratory for Novel Software Technology. Her research interests include distributed computing, wireless networks, and pervasive computing. She has published over 80 papers in refereed journals and conferences in the above areas. She is a member of IEEE.



Yu Liang is a lecturer at Nanjing Normal University. She received the MS and PhD degrees from Nanjing University in 2011 and 2021, respectively. She was a senior software engineer in Trend Micro China Development Center between 2011 and 2017. Her research interests include edge intelligence and edge computing. Her publications include those appeared in TMC, TPDS, TON, Computer Networks, Computer Communications, IEEE ICDCS, IEEE MSN, and IEEE Globecom.



Qing Gu is currently a Professor and Ph.D. Advisor with the Department of Computer Science and Technology, Nanjing University, Nanjing, China, and a Member of the National Key Laboratory of Novel Software Technology. His research interests include natural language processing, machine learning, and software engineering.